

UPSPA IMPLEMENTATION NOTES

Report II in the series • Report I (May 2026): Storage-Provider cryptography and deployment

Updatable Single Password Authentication: client-side policy detection and deterministic password derivation in the browser extension

An implementation and testing report

Efe Bekteş

Istanbul Technical University, Department of Computer Engineering
Summer intern, CRYPTO@KU, Koç University • bektes23@itu.edu.tr
June 2026 • github.com/rezasaadi/UpSPA_extension

Abstract. UpSPA is a browser-based password manager that realises Threshold Single Password Authentication: from one master password the client re-derives a unique, high-entropy value for every website, while the master password is never learned by the login server, by any storage provider, or by an adversary who corrupts fewer than a threshold of the providers. The derived value is produced in the browser extension, which is where it meets a real web form. For that to work in practice two client-side problems must be solved: the generated password must *satisfy the site's password policy* so the site accepts it, and it must be *deterministically reproducible* at every later login. This report documents two contributions to the Chrome extension (`packages/extension`). The first (Task 3) is a robust, unit-tested password-policy *detector* that infers a site's requirements from the password input's constraints, its `pattern` attribute, and the surrounding helper text, with explicit handling of length ranges, false-positive suppression, listed allowed-symbol sets, and lookahead-aware pattern parsing. The second (Task 6) is a deterministic password *encoder*, $\text{Password}_i = \text{Encode}(vInfo, \text{Counter}, \text{PasswordPolicy})$, with a clean rotation counter, structured-or-JSON policy input, and persistence of the policy alongside each per-site record. We describe both, their integration with the popup and options pages and the per-origin account record, give a defensive security and privacy review, and report a test suite of 34 cases (21 detection + 13 encoder) that passes alongside the existing client tests, with the whole system brought up end-to-end on a fresh Windows host via WSL. The HTTP/API, database, and Storage-Provider cryptography layers are the work of other contributors; the focus here is the assigned extension work and its verification. This report is the second in a short series; Report I [1] covers the Storage-Provider cryptography and the end-to-end deployment, and is assumed as background rather than repeated.

Keywords: single password authentication; password managers; password policy; deterministic password generation; oblivious pseudorandom function; browser extension.

1 Introduction

A traditional password manager stores a user's passwords. UpSPA takes a different route: the user remembers a single *master password*, and from it the system re-derives a unique, high-entropy secret for every website. That derivation is distributed across several independent storage providers (SPs) so that, by construction, no single party ever sees the master password. This is the Threshold Single Password Authentication scheme of İşler and Küpçü [3], generalising the single-provider construction of Acar et al. [2]; the repository implements it as a Rust/WebAssembly core, a TypeScript client, a Chrome extension, and a set of Go Storage Providers backed by PostgreSQL.

Report I [1] covered the Storage-Provider cryptography and the end-to-end deployment of this same system; its scheme summary (§2), architecture (§3), and protocol phases (§4) are the background for this report and are not repeated here. The present work is on the other end of the system, in the browser extension, where the derived value must actually be typed into a website's password field. Two practical gaps sit exactly there. First, websites impose *password policies* (a minimum length, required character classes, an allowed symbol set, and so on); a derived password that violates the policy is rejected at the form, no matter how sound the cryptography behind it. The

Table 1: Extension-side names used in this report (the protocol notation is in Report I, Table 1).

Term	Meaning
<i>vInfo</i>	The per-site value reconstructed from the threshold-OPRF state key; the encoder’s secret input. The master password never leaves the browser.
PasswordPolicy	Normalised site requirements (length bounds, required classes, allowed symbols, whitespace, forbidden substrings). <code>packages/extension/src/shared/passwordPolicy.ts</code> .
Counter	A per-site rotation index. Rotating a site password means incrementing it.
Detector	Pure logic that infers a policy from page signals. <code>policyDetection.ts</code> (new).
Encoder	<code>Encode(vInfo, Counter, Policy)</code> . <code>encodeSecretAsPassword</code> in <code>passwordPolicy.ts</code> .
siteAccounts	Per-origin record store holding <code>accountId</code> , <code>passwordPolicy</code> , <code>encoderCounter</code> .

extension’s policy detector existed but was only partially functional: it read a few input attributes and a narrow set of phrasings and missed common ones. Second, the per-site password is produced by a deterministic *encoder* from the reconstructed value; the encoder existed but conflated its rotation counter with an internal retry index, so the counter a caller stored could silently differ from the one it asked for. This report addresses both, scoped to the two assigned tasks:

- **Task 3: Password policy detection.** Make the detector reliably infer a site’s requirements across the ways real pages express them, and make its core logic pure and unit-testable.
- **Task 6: Deterministic password encoder.** Realise $\text{Password}_i = \text{Encode}(vInfo, \text{Counter}, \text{Policy})$ with a clean rotation counter, accept a structured or JSON policy, and persist the policy with each site record.

Section 2 recalls just enough of the scheme to place the work. Section 3 describes the extension’s internal architecture and the detect $\rightarrow$ encode $\rightarrow$ fill $\rightarrow$ store data flow. Sections 4 and 5 are the two contributions. Section 6 shows the integration with the popup and options pages. Section 7 is a defensive security and privacy review, Section 8 the test suite, and Section 9 the end-to-end run and reproduction. Section 10 states the scope and limitations, and Section 11 concludes.

2 Background and the per-site record

Report I [1] develops the Threshold SPA scheme [3, 2] in full: its Table 1 fixes the protocol notation and its §4 walks through every phase. We reuse those names and recall only the one fact this report needs. After a threshold-OPRF exchange the client reconstructs a per-site value *vInfo*, bound to the login-server domain *ls* so that a look-alike domain yields a different value, and Report I submits *vInfo* to the website in place of a password. The contribution of *this* report sits one step later: turning *vInfo* into a concrete password the site will accept and that the extension can reproduce on every visit.

Two objects. The extension layer turns on two objects that Report I does not detail.

The password policy. A normalised description of what the site requires: a minimum and maximum length, which of the four character classes (upper, lower, digit, symbol) are required, the allowed symbol set, whether whitespace is forbidden, and any forbidden substrings (such as the username). It may be carried as a structured object or, equivalently, as JSON.

The per-site record. For each site the extension keeps a record $\langle \text{WebsiteURL}, \text{SUID}, \text{UID}, \text{PasswordPolicy} \rangle$. In the extension this is realised per browser origin: `siteAccounts` maps an origin to one or more accounts, each carrying its `accountId`, the detected `passwordPolicy`, and an `encoderCounter`. Persisting the policy with the record is what lets the encoder regenerate exactly the right password for a given user and site later.

Table 1 fixes the few names used below.

3 The extension in the system

The overall five-component architecture is in Report I (§3, Fig. 1); here we zoom into the Chrome extension. Within it, three parts cooperate (Fig. 1). A *content script* runs in the page, reads the password field and its surrounding text, and fills the form. A *popup* and an *options* page provide the user interface: options provisions the SPs once; the popup detects the policy, derives the password, and triggers the fill. The extension calls the WebAssembly crypto core (via the `upspa-js` client) for the protocol and speaks HTTP to the SPs. The two contributions of this report are

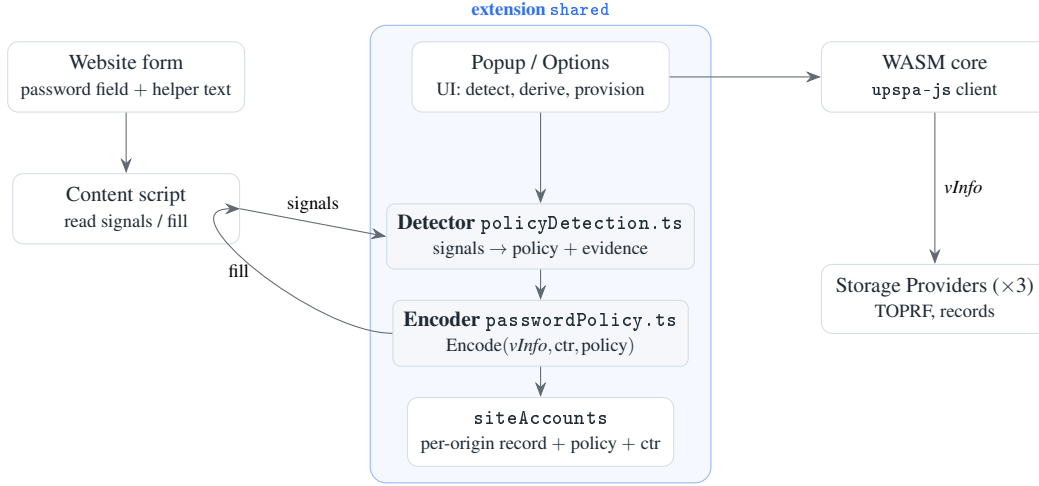


Figure 1: The extension internals. The content script gathers page signals and fills the form; the popup/options derive the password and provision the SPs. The two boxed modules are the contributions of this report.

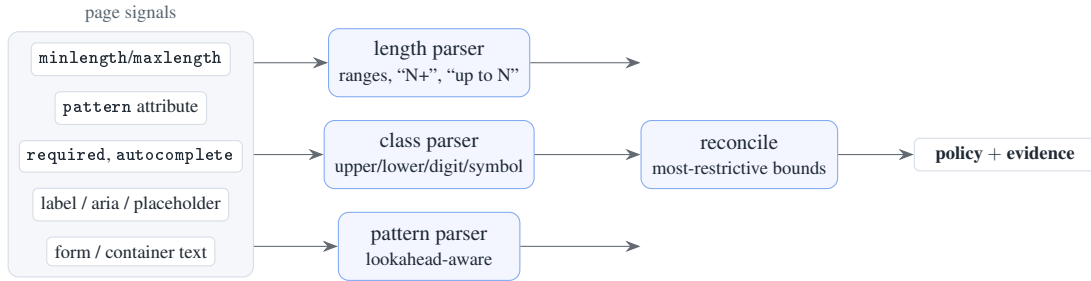


Figure 2: The detection pipeline. Page signals feed three pure parsers; the results are reconciled into a single policy with the most-restrictive consistent bounds, plus an evidence trail the user can review.

the policy *detector*, whose pure logic is exercised by the content script, and the deterministic *encoder*, called from the popup; both sit in the extension’s shared layer.

The end-to-end flow for using a site is: (1) the content script gathers the password field’s attributes and nearby text; (2) the detector turns those into a policy and human-readable evidence; (3) the user reviews the policy in the popup; (4) the encoder derives the site password from *vInfo*, the counter, and the policy; (5) the content script fills it; (6) the policy and counter are persisted with the per-origin record so the same password regenerates on the next login.

4 Task 3: Password policy detection

4.1 The problem

Websites state their password rules in inconsistent ways: as HTML constraints (*minlength*, *maxlength*, *pattern*, *required*), and as free text near the field (“at least 8 characters”, “must include an uppercase letter and a number”, “allowed special characters: !@#\$”). The previous detector lived inline in the content script and handled only a few of these. Concretely it missed length *ranges* (“8–64 characters”, “between 8 and 20”); it read the bare word *number* as a digit requirement, so “phone number” produced a false positive; on any symbol hint it hard-coded the allowed set to !@#%\$%^&*, discarding a site’s actual list; it did not handle “spaces are allowed”; and it parsed only the first quantifier of a *pattern*, treating the trailing body class (which merely *lists* allowed characters) as if those classes were *required*.

4.2 Design

The parsing logic is extracted into a new module, `policyDetection.ts`, with no DOM or chrome access, so it can be unit-tested directly. The content script’s only remaining job is to gather *signals* from the live page and hand them to the pure entry point `detectPasswordPolicy(signals)`, which returns a partial policy plus an evidence list. The signal sources and the parsing stages are shown in Fig. 2.

Table 2: Representative detection results (abbreviated; ✓ marks a required class).

Input signal	Length	U	L	D	S	Symbols
“Must be at least 8 characters.”	min 8					
“Password must be 8–64 characters long.”	8–64					
“between 8 and 20 characters”	8–20					
“upper, lower, a number, special char”	n/a	✓	✓	✓	✓	default
“Enter your phone number to continue.”	n/a					(no digit)
“allowed special characters: !@#\$”	n/a				✓	!@#\$
minlength=10, text “Maximum 32 characters”	10–32					
lookahead pattern (Fig. 1)	min 8	✓	✓	✓	✓	@\$!%*?&

Listing 1: A lookahead pattern and what the detector extracts. Only the lookahead classes are treated as required; the trailing body class lists the allowed alphabet; the symbol-only lookahead becomes the allowed set.

```
pattern = ~(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$
=> requireUpper, requireLower, requireDigit, requireSymbol
    allowedSymbols = @$!%*?&    minLen = 8    maxLen = (none)
```

Length. The length parser recognises a wide set of phrasings and collects every candidate minimum and maximum: “at least N”, “minimum of N”, “no fewer than N”, “N+”, “N or more”, “up to N”, “at most N”, “no more than N”, and ranges written “8–64”, “8 to 32”, or “between 8 and 20”. The final minimum is the *largest* candidate minimum and the final maximum the *smallest* candidate maximum, so the chosen password fits every stated bound; if the two conflict (a contradictory page) they collapse to a fixed length, recorded in the evidence.

Character classes. Uppercase and lowercase are matched directly. The digit rule is the one most prone to false positives, so phrases like “phone/account/card/order number” are stripped before testing for “number” or “digit”. For symbols, an explicitly *listed* allowed set is captured (e.g. “allowed special characters: !@#\$” yields !@#\$), restricted to a “listable” alphabet that excludes ordinary prose punctuation so a descriptive colon or comma is not mistaken for a symbol; absent a list, a generic symbol requirement falls back to the default set. “Spaces are allowed” clears the forbid-whitespace flag, and “no spaces” sets it.

Pattern. The pattern attribute is parsed with awareness of lookaheads. When the pattern uses lookaheads, only classes inside a $(?=.*\dots)$ group are treated as *required*, while the trailing body class is understood to merely *list* the allowed alphabet; a symbol-only lookahead class is captured as the allowed set. Length is taken from the most restrictive $\{n,m\}$ bounds found anywhere in the expression. Without lookaheads the parser falls back to a whole-pattern scan.

Reconciliation and evidence. Candidates from the attributes, the pattern, and the text are merged: booleans are OR-ed (any source requiring a class wins), length bounds are reconciled to the most restrictive consistent values, and the listed symbol set, if any, overrides the default. Every decision pushes a short, human-readable note; the popup shows the first few so the user can review or correct the policy before registering. Table 2 gives representative inputs and the policy the detector produces.

5 Task 6: Deterministic password encoder

5.1 The contract

The encoder realises

$$\text{Password}_i = \text{Encode}(\text{vInfo}, \text{Counter}, \text{PasswordPolicy}),$$

implemented as `encodeSecretAsPassword` in `passwordPolicy.ts`. Determinism is the property that matters: for fixed inputs the same password must come out, so that registration and every later login derive the identical value. The *Counter* is a per-site rotation index chosen by the caller; rotating to a new password means incrementing it.

5.2 Construction

Given the normalised policy, the encoder fixes a deterministic target length within the policy bounds and a character pool drawn from exactly the required classes (and the allowed symbol set when symbols are required). It then derives

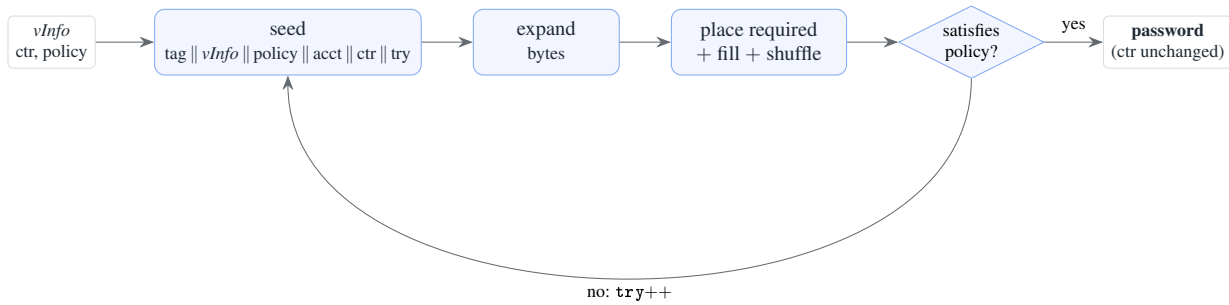


Figure 3: The deterministic encoder. Rejection sampling happens *within* a fixed counter via an internal attempt index, so the returned counter always equals the requested one and register/login reproduce the same password.

Listing 2: The core of the encoder (abridged). The counter is fixed; only the internal attempt index advances on a re-draw, so the returned counter equals the requested one.

```

for (let attempt = 0; attempt < MAX_ATTEMPTS; attempt += 1) {
  const seed = [ 'upspa-password-encoding-v2', secretB64,
    canonicalPolicy(policy), accountKey,
    'ctr=${counter}', 'try=${attempt}' ].join('|');
  const bytes = await expandBytes(seed, length * 2 + required.length);
  // one char per required class, then fill from the pool, then shuffle:
  const password = buildCandidate(chars, bytes.slice(length, length * 2));
  if (passwordSatisfiesPolicy(password, policy, accountId))
    return { password, counter }; // counter == requested counter
}
throw new Error('Could not encode a password that satisfies the site policy.');
```

a byte stream by hashing a domain-separated seed that binds the version tag, *vInfo*, the canonical policy, the account identifier, the counter, and an internal attempt index; from those bytes it places one character from each required class, fills the remaining positions from the pool, and applies a deterministic Fisher–Yates shuffle so the required characters are not positionally predictable. The candidate is checked against the policy (including any forbidden substrings); if it fails, the attempt index advances and the stream is redrawn. The stages are shown in Fig. 3 and the core in Listing 2.

5.3 What changed

Three corrections and additions make the encoder match its contract; none changed the function’s signature, so the popup that calls it needed no edits.

1. **Clean counter semantics.** Previously, on a candidate that failed the policy the encoder advanced the counter itself (*candidate* = counter + attempt) and returned the advanced value, so a caller that stored the result could drift away from the rotation index it asked for. The encoder now resamples *within* a fixed counter using an internal attempt index and always returns the requested counter; the seed binds both, so determinism is preserved while the counter stays a clean, caller-controlled rotation index.
2. **Structured or JSON policy.** A small `parsePasswordPolicy` accepts the policy either as an object or as a JSON string and normalises it, matching the specification’s note that a policy may be carried as text or JSON; invalid JSON raises a clear error.
3. **Persistence with the record.** The detected policy and the counter are stored with the per-origin record in `siteAccounts`, realising the $\langle \text{WebsiteURL}, \text{SUID}, \text{UID}, \text{PasswordPolicy} \rangle$ requirement on the extension side, so the encoder can regenerate the right password on a later login.

The seed’s version tag was advanced from v1 to v2 to reflect the change in derivation; this deliberately changes the derived passwords, which is safe for the current pre-release demo but means any demo accounts created against the old encoder must be regenerated.

6 Integration and the extension UI

The two modules are reached through the existing user interface; no message-passing contracts changed (Report I, §9, shows the login server’s own pages; the figures here are the extension’s popup and options). The options page (Fig. 5)

UpSPA
Current tab origin
http://localhost:3000

Known accounts
alice

Add or edit account ID
alice

Add / update account

Delete selected account

Master password

Detect password policy

Password policy

Min
12

Max
64

☒ Uppercase
☒ Lowercase
☒ Digit
☒ Symbol
☒ Forbid whitespace

Allowed symbols
!@#\$%^&*

Forbidden substrings
username, email

Counter 0: Evidence: detected minimum length 12 | detected uppercase requirement | detected digit requirement | detected symbol requirement

Register current site

Generate another candidate

Confirm Registration Success

Login current site

Prepare secret update

Commit secret update after website success

Cancel prepared secret update

Password policy detected. Review or edit before registration.

Lock extension

Open Options

Figure 4: The extension popup, rendered from the project’s `popup.html` with representative detected data: the detected policy (Min 12, Max 64, required classes, allowed symbols) and the evidence line are shown for review before the encoder derives and fills the password.

UpSPA Options

Setup / Provision

Global UpSPA UID
test-user

Master Password

Threshold
2

Storage Providers
1,http://localhost:8081
2,http://localhost:8082
3,http://localhost:8083

Save Config Only

Setup / Provision SPs

Master Password Update

Old Master Password

New Master Password

Confirm New Master Password

Update Master Password

Lock Extension Session

Status

Setup/provision completed successfully.

Figure 5: The options page, rendered from `options.html`, provisioned for the 2-of-3 demo: the global UID, master password, threshold, and the three Storage Providers, with a successful setup status.

provisions the SPs once. The popup (Fig. 4) is where detection and derivation are exercised: *Detect password policy* runs the content script and fills the policy panel with the detected values and an evidence line; the user reviews or edits them; *Register current site* derives the password with the encoder and fills the form; *Generate another candidate* increments the counter to rotate; *Login current site* re-derives the identical value. The two figures below are rendered directly from the extension’s own `popup.html` and `options.html` with representative detected data; they show the genuine markup and CSS, and a live capture from the running extension can be dropped in their place (see §9).

7 Security and privacy review (defensive)

The properties of the threshold construction are inherited from the scheme [3, 2] and the Storage-Provider review in Report I (§10); the points below concern what the extension contribution itself adds or must not undermine. The intent is to harden the prototype.

- **The derived password reveals nothing about $vInfo$.** The encoder maps $vInfo$ through a domain-separated, hashed expansion; the site sees only the final password, which is per-site and useless elsewhere. Determinism is reproducibility, not a weakness: the secret input is the high-entropy state-key value, not a guessable seed.
- **The counter is a rotation index, not a secret.** Making it caller-controlled removes a latent footgun in which a stored counter could silently diverge from the one requested, which would have made a previously generated password irreproducible.

Listing 3: The extension and client test suites after the contribution.

```
$ ( cd packages/extension && npx vitest run )
ok src/shared/policyDetection.test.ts (21 tests)
ok src/shared/passwordPolicy.test.ts (13 tests)
Test Files 2 passed (2)
Tests 34 passed (34)

$ npm -w upspa-js run test
ok test/upspaClient.test.ts (3 tests)
ok test/base64url.test.ts (2 tests)
Tests 5 passed (5)
```

Table 3: Test inventory for the contribution.

Suite	Tests	Coverage
policyDetection.test.ts	21	length phrasings/ranges, digit false-positive suppression, listed/default symbols, whitespace directions, forbidden substrings, lookahead and plain patterns, merge/conflict.
passwordPolicy.test.ts	13	determinism, distinct counters, length/class compliance, forbidden-substring avoidance, impossible-policy error; counter stability, register=login reproduction, JSON policy, invalid-JSON error, fixed length, restricted symbol set.
upspa-js (unchanged)	5	client and base64url round-trips; no regression.

- **Detection is best-effort and user-reviewable.** The detector errs toward *over*-requiring a class rather than under-requiring, because a password that includes an extra allowed class is still accepted whereas a missing required class is rejected; the popup always shows the detected policy and evidence for the user to correct before anything is sent.
- **Policy is non-secret, but the site↔account map is privacy-sensitive.** The per-origin record (origin, account id, policy, counter) is kept in `chrome.storage.local` in the clear. The policy itself is not secret, but the mapping of which accounts exist on which sites is. Encrypting that map under the master password is the natural place for the sibling task on local protected storage; it is called out here so the two tasks stay consistent.

Named, not tested. Two items are stated rather than left for a reviewer. The byte selection uses a modulo reduction over the character pool, which carries a small, classical modulo bias; given the entropy of the state-key input this is not a practical weakness for a password, but it is noted rather than claimed uniform. And the detector is heuristic over natural-language hints; it is designed to be corrected by the user, not to be a complete parser of every phrasing.

8 Testing and verification

The two modules are covered by a unit suite that runs under Vitest. Detection has 21 tests spanning the length phrasings and ranges, the digit false-positive suppression, listed and default symbol sets, the whitespace directions, the username/email forbidden substrings, the lookahead and plain pattern forms, and the end-to-end merge with conflicting bounds. The encoder has 13 tests: the pre-existing guarantees (determinism, distinct counters give distinct passwords, length and class compliance, forbidden-substring avoidance, impossible policy raises) plus the new ones for this task: the returned counter equals the requested one, register and login reproduce the same password, a JSON-string policy equals the object form, invalid JSON is rejected, a fixed-length (min = max) policy yields the exact length, and a restricted allowed-symbol set is honoured. The full extension suite passes 34/34, and the existing upspa-js suite passes 5/5 unchanged (Listing 3). A type-check of the changed files is clean; the only `tsc` errors in a fresh tree are the pre-existing upspa-js module-resolution notes in untouched files, which disappear once the WASM → JS build has run.

9 End-to-end run and reproduction

Report I (§11) reports a browser-free run of all seven protocol phases (7/7); this section exercises the same flow in the browser through the popup. The whole system was brought up on a fresh Windows 11 host. Because the demo scripts are Linux shell (they use `tmux` and `pg_isready`, and even print WSL-IP URLs for Windows Chrome), the backend and build run inside WSL2 (Ubuntu) and Chrome runs natively on Windows against the WSL backend. A single setup

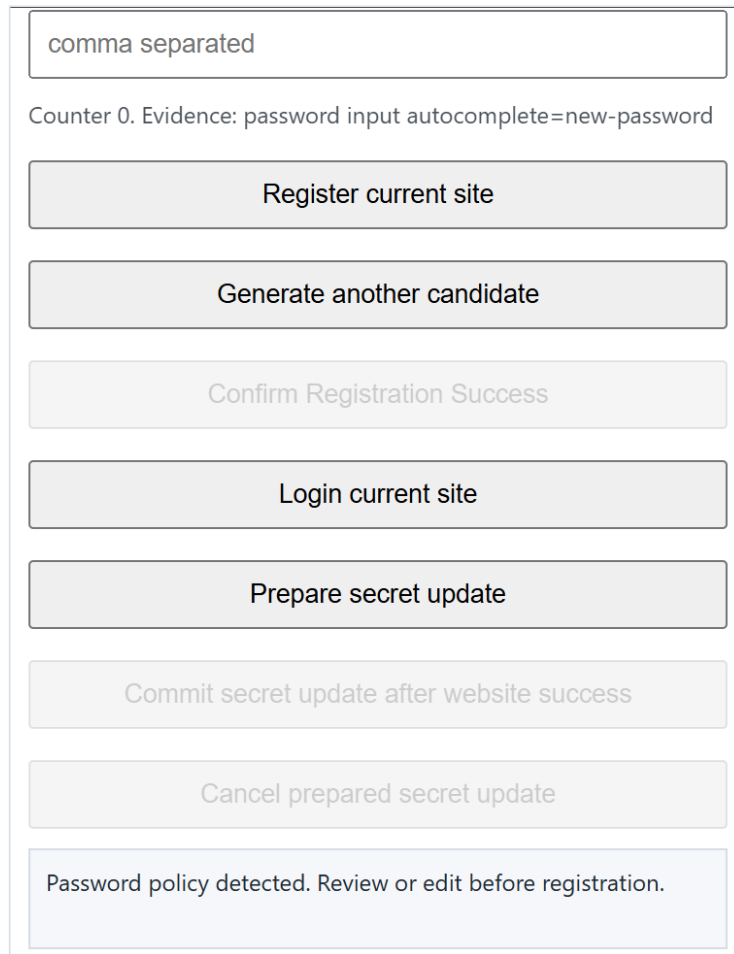


Figure 6: Live in-browser capture (Chrome on Windows) of the popup immediately after *Detect password policy* on the demo registration page. The detector reports its evidence (`password input autocomplete=new-password`) and the “Password policy detected. Review or edit before registration.” status, with the registration and login actions below. The same run went on to register and log in successfully, confirming the encoder reproduced the per-site value deterministically.

script installs the toolchains (Node, Go, Rust with `wasm-pack`, PostgreSQL), creates the demo role and databases, builds `WASM` $\rightarrow$ `JS` $\rightarrow$ extension, and starts the three SPs and the login server; the assigned changes are then applied to the working tree and the extension is rebuilt. The Task 3 and Task 6 changes documented here are committed on the `intern/extension` branch (base `v0.1_local`) of the repository. One environment note worth recording: on a fresh host `wasm-pack` could not fetch the binaryen optimiser from GitHub, so `wasm-opt` is disabled in the `WASM` crate’s metadata (a one-line, local-only build setting); the WebAssembly is otherwise identical. The build then completes and the demo runs.

Against the demo login server the popup flow exercises the contribution directly: on the registration page, *Detect password policy* runs the Task 3 detector and shows the inferred policy and evidence; *Register* derives the password with the Task 6 encoder and fills the form; and a subsequent *Login* re-derives the identical value, which is the visible confirmation that the counter fix preserves reproducibility. A live in-browser capture of this run is shown in Fig. 6.

On the figures. The popup and options figures in §6 are rendered from the extension’s own `popup.html` and `options.html` so the full layout is visible at once; Fig. 6 is a genuine in-browser capture from the live run. (The popup is taller than one screen and, opened as a standalone tab, reports itself as the active tab, so a single full-window capture of a live detection is awkward; the rendered figures therefore stand in for the complete layout.)

10 Limitations and scope

Three points are stated plainly. First, the detector is a best-effort inference over how pages state their rules; it is deliberately conservative (over-requiring rather than under-requiring) and user-correctable, not a complete natural-language parser. Second, the encoder’s byte selection carries a small modulo bias that is immaterial at the entropy of

the state-key input but is not claimed to be perfectly uniform. Third, this report’s verification is the unit suite plus the end-to-end demo run; it is functional verification on the demo login server, not a study against a large corpus of real-world signup pages, which is the natural next measurement. None of these are correctness defects in the delivered work.

11 Conclusion

The extension now reliably infers a site’s password policy from the several ways pages express it, and derives a per-site password that satisfies that policy and reproduces deterministically at every login. The detection logic is pure and unit-tested; the encoder has clean rotation semantics, accepts a structured or JSON policy, and persists the policy with each per-site record. The suite passes 34/34 for the contribution with no regression in the client tests, and the whole system runs end-to-end on a fresh host. What remains is engineering toward the project’s next steps: auto-fill of the broader sign-in form under the master password, and the move to real Storage-Provider deployments and a mobile/desktop refactor, not the assigned client logic, which is complete and tested.

Note on tools. This report and the accompanying code were prepared with AI assistance; all claims, code, and test results were re-checked against the repository and against real runs before use. The UI figures are rendered from the extension’s own HTML and CSS and are labelled as such, not presented as live screen captures.

Acknowledgments

The author thanks the UpSPA project lead, Hamidreza Saadi, for the task and guidance, and Prof. Alptekin Küpçü and Assoc. Prof. Aykut Coşkun for hosting the internship in the CRYPTO@KU group. The HTTP/API, database, and Storage-Provider cryptography layers are the work of other team members.

References

- [1] E. Bekteş. *Updatable Single Password Authentication: deployment, Storage-Provider cryptography, and a browser-free demo*. UpSPA Implementation Report I, CRYPTO@KU internship, May 2026.
- [2] T. Acar, M. Belenkiy, A. Küpçü. Single Password Authentication. *Computer Networks* 57(13):2597–2614, 2013.
- [3] D. İşler, A. Küpçü. Threshold Single Password Authentication. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology* (ESORICS 2017 Workshops, DPM/CBT), LNCS 10436, Springer, 2017.
- [4] A. Davidson, A. Faz-Hernández, N. Sullivan, C. A. Wood. Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups. RFC 9497, 2023.
- [5] H. de Valence, J. Grigg, G. Tankersley, F. Valsorda, I. Lovecruft, et al. The ristretto255 and decaf448 Groups. RFC 9496, 2023.
- [6] H. Krawczyk, P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.
- [7] S. Josefsson. The Base16, Base32, and Base64 Data Encodings. RFC 4648, 2006.
- [8] P. A. Grassi et al. Digital Identity Guidelines: Authentication and Lifecycle Management. NIST SP 800-63B, 2017.